I. WSAN Analyzer GUI: Functional Specification

A. Purpose

The *WSAN Analyzer GUI* is a desktop application that analyzes a Cooja radio communication log (`Mote_Communication.txt`) to compute protocol metrics (reliability, stability, overhead) for the WSAN protocol and to visualize per-message dissemination as an interactive friend-ACK tree.

B. Input Data and Line Format

1) Single input file: The application accepts exactly one input file selected by the user via the `[Open Log File]` button: `Mote_Communication.txt`.

2) Expected line format: Each log line must contain at least four tab-separated fields:

1) `time_ms`: simulation time in milliseconds (used as the main time axis).
2) `src_mote_id`: sender mote ID as reported by the Cooja log (not necessarily equal to WSAN `SenderID`).
3) `receivers`: either a comma-separated list of receiver mote IDs (e.g., `"6,18,30"`) or `"-"` for broadcast.
4) `len: hex_dump`: a length prefix followed by a hex dump. The hex dump begins after the first colon `:`.

C. WSAN Frame Detection and Filtering

1) Hex dump normalization: Before searching for WSAN frames, the application normalizes the hex dump:

- keep only the substring after `:`,
- remove any `0x`/`0X` prefix,
- remove whitespace (spaces, tabs, newlines),
- convert to uppercase.

This normalization is required because the WSAN signature may appear split by whitespace in the raw dump.

2) WSAN signature and extraction: A WSAN frame is considered present if the normalized dump contains the ASCII signature `"WSAN"`, i.e., `5753414E` in hex. The application extracts the WSAN frame starting at the first occurrence of `5753414E`, using the WSAN `MsgLen` field to determine the frame length. The checksum test (CS/Chk8) is deliberately disabled and must not discard frames.

3) Network Load definition: The *Network Load* graph and related statistics must be computed only from successfully extracted WSAN frames (i.e., lines where `5753414E` was found and a WSAN frame was extracted). Non-WSAN radio frames are ignored for Network Load.

D. WSAN Field Decoding

After extraction, the application decodes the following WSAN fields (wire format):

- `Marker` (3 bytes): `"Mbl"` or `"RSU"`,
- `SenderID` (u32, big-endian): physical sender (changes per hop),
- `MsgType` (u8): `QUERY=0`, `ACK=1`, `DATA=2`, `EMERGENCY=4`,
- `MsgID` (u32, big-endian),
- `TargetID` (u32, big-endian),
- `OriginID` (u32, big-endian): creator (stable),
- `Payload` (variable length),
- `TimeStamp` (u32, big-endian).

The time axis for analysis is `time_ms` from the first column of the log, not the WSAN `TimeStamp` field.

E. Frame Validity Rules (False Positive Protection)

To avoid false positives where the sequence `5753414E` appears inside unrelated payloads, the application applies an ID validity filter:

- `SenderID` and `OriginID` must lie within the expected mote ID range (default: `0..255` for Sky/`node_id8`).

Frames failing this constraint are ignored for metrics and for tree building.

F. Message Identity and Core Definitions

1) Message key: A logical message is identified by:

$$\texttt{MessageKey} = (\texttt{OriginID}, \texttt{MsgID})$$

2) Created vs. Seen:

a) Created (strict).: A message is **Created** iff there exists at least one WSAN frame for that `MessageKey` such that:

- `MsgType` is `DATA` or `EMERGENCY`, and
- `SenderID == OriginID`.

`TimeCreated` is the `time_ms` of the earliest such frame.

b) Seen (observed).: A message is **Seen** if any WSAN `DATA`/`EMERGENCY` frame for that `MessageKey` exists in the log, regardless of `SenderID`.

c) Application policy.: The Messages table and the core reliability/stability metrics must be computed from **Created messages only**. Any `MessageKey` that is Seen but not Created (i.e., forwarded without a creation point present in the log) must be ignored in the main message list and derived metrics.

3) Delivered to RSU (strict): A message is **Delivered** iff an RSU ACK confirming that message exists:

- WSAN frame has `Marker == "RSU"` and `MsgType == ACK`,
- ACK payload contains `acked_type`, `acked_origin`, `acked_msg`,
- `acked_origin == OriginID` and `acked_msg == MsgID`.

`TimeDelivered` is the `time_ms` of the earliest such RSU ACK.

G. Hop Count for DATA

For `DATA` messages, hop count is computed using the TTL field in the `DATA` payload:

- `ttl8` is stored in the `DATA` payload (last byte of the `wsan_data_payload_t` structure),
- `TTL_DEFAULT = WSAN_DATA_TTL_DEFAULT` (default 3).

For delivered DATA, the hop count is:

$$\texttt{HopCount} = \texttt{TTL\_DEFAULT} - \texttt{ttl8\_at\_RSU}$$

where ttl8_at_RSU is taken from the last DATA transmission to the RSU immediately preceding the RSU ACK.

H. User Interface Requirements

1) Messages Tab: The Messages tab displays one row per **Created message** with at least:

- TimeCreated,
- TimeDelivered,
- Latency = TimeDelivered − TimeCreated (if delivered),
- OriginID,
- MsgID,
- Type (DATA / EMERGENCY),
- Delivered (True/False),
- HexNorm: normalized WSAN frame dump for the creation frame, stored as frame.hex().upper().

2) Search/Filter: The Messages tab provides a search/filter input that filters rows by substring match on OriginID and/or MsgID.

3) Log Stats Tab: A dedicated **Log Stats** tab presents aggregate statistics computed from the log, including at minimum:

- total lines,
- total WSAN frames found/extracted,
- WSAN frames by marker (Mbl/RSU),
- WSAN frames by type (QUERY/ACK/DATA/EMERGENCY),
- unique DATA keys Seen,
- unique keys Created (strict),
- Seen-but-not-Created count,
- Delivered (RSU ACK) count.

4) Graphs Tab: The Graphs tab presents:

- PDR over time (Delivered/Created per time bin),
- delivered throughput over time (delivered messages per bin),
- latency CDF (Created → RSU ACK),
- hop count histogram (delivered DATA),
- Network Load over time (WSAN frames/sec and WSAN bytes/sec),
- per-mobile Created vs Delivered bar chart.

Time bin size is controlled by a slider with range 10–120 seconds, default 60.

I. Streaming Parse, Progress, and Watchdog

- Parsing must be performed in a background thread to keep the GUI responsive.
- A progress bar must reflect parsing progress.
- A watchdog dialog must appear every 60 seconds of wall-clock parsing time (unless "Don't ask again" is checked), asking whether to continue or stop the operation.

J. Friend-ACK Tree Visualization

1) Tree invocation: In the Messages tab, right-clicking a message row opens a context menu action Show distribution tree.

2) Tree scope policy: Trees are generated only for messages that are present in the Created message table. If a MessageKey has no creation point in the log, tree visualization must be refused with a user-facing notice.

3) On-demand tree construction: To avoid storing large structures for the entire log, the tree is constructed **on demand** by scanning the log again and collecting only events relevant to the selected MessageKey:

- DATA frames: time, sender, friend list (friend1, friend2),
- mobile ACK frames: time, ACK sender, and referenced acked_origin/acked_msg,
- RSU ACK: first delivery confirmation time; infer last_hop_to_RSU as the last unicast DATA before that ACK.

4) Edge rules: A dashed directed edge Parent → Child is added if:

- a DATA frame from Parent lists Child in its friend list, and
- Child sends an ACK for the same MessageKey within a time window ack_window_ms (default 2000 ms) after the relevant DATA transmission.

By default, the visualization uses a first-parent tree policy (each child keeps its first discovered parent). An optional "Show all edges" mode may display all valid edges (DAG).

5) RSU delivery edge: If the message is Delivered, the RSU is displayed as a larger node, and the edge last_hop_to_RSU → RSU is drawn as a bold solid line. If the message is not Delivered, the RSU is not shown and all edges remain dashed.

6) Tree window interaction: The tree is rendered using QGraphicsView/QGraphicsScene and supports:

- zoom with mouse wheel,
- panning by mouse drag.

K. Export

- Export Messages and per-mobile summaries as CSV.
- Export plots as PNG.